EthReliever: ソースコードの構造的特徴を考慮した大規模言語モデルによるスマートコントラクトの脆弱性修正

今井 泰正[1,a)]　矢内 直人[1]

概要：Ethereum スマートコントラクトに対する攻撃による被害が多数報告されている一方、それらのソースコードの構造的特徴がスマートコントラクトの脆弱性の修正に寄与するかは非自明である。本稿では、グラフニューラルネットワーク（GNN）と大規模言語モデル (LLM) を通じてソースコードの構造的特徴を活用することで、脆弱性を含むスマートコントラクトを自動修正する手法 EthReliever を提案する。EthReliever では GNN がソースコードから脆弱性の原因箇所を分析し、ソースコードとその分析結果を LLM へ入力することで、脆弱性を修正したソースコードを生成する。EthReliever の性能を実験評価したところ, 既存手法と比べて脆弱性の修正が大幅に改善されることを確認した。これは LLM が脆弱性を修正する際にソースコードの構造的特徴がどういう形式で与えられるかが重要であり、EthReliever では GNN による抽出が LLM による脆弱性の修正に寄与したためである。また、GNN を活用することで脆弱性修正に掛かるトークンの消費を抑えることができることを示す。とくに脆弱性の修正時に EthReliever が内包する LLM の処理を繰り返し実行することで、高い確率で脆弱性を修正できる。

キーワード：スマートコントラクト, 脆弱性修正, 大規模言語モデル, グラフニューラルネットワーク

Smart Contract Vulnerability Repair Based Graph Aligned Large Language Models

Yasumasa Imai[1,a)]　Naoto Yanai[1]

Abstract: There have been many reports of damages caused by attacks on smart contracts that contain vulnerabilities. In this paper, we propose a method to automatically repair smart contracts that include vulnerabilities by leveraging graph neural networks (GNNs) and LLMs. In the proposed method, a GNN is used to identify the root cause of vulnerabilities from the Abstract Syntax Tree (AST) of the source code. Then, the source code and the information about the identified vulnerability are input to an LLM to generate corrected code that eliminates the vulnerability. We implemented the proposed method and evaluated it using a Solidity smart contract dataset. Additionally, an ablation study demonstrates that identifying the root cause with a GNN is effective in improving the LLM's code repair performance.

Keywords: Smart Contract, Vulnerability Repair, Large Language Model, Graph Neural Network

1. 序論

1.1 背景

Ethereum スマートコントラクト [1] は、仮想通貨 Ether を対価にブロックチェーン上で実行される自己実行型のプログラムであり、暗号資産の管理や分散型金融（DeFi）、NFT などの様々なサービスで利用されている [2]。一方で、スマートコントラクトに含まれる脆弱性を悪用した被害が多数報告されている。実際に 2025 年には、Ethereum 上でハッキングや詐欺を含む脆弱性の悪用による被害が約 310 件発生し、被害総額は約 16 億米ドルに達しており [3]、脆弱性の悪用は現在も継続する深刻な課題である。代表例と

[1] パナソニック ホールディングス株式会社
Panasonic Holdings Corporation
[a)] imai.yasumasa@jp.panasonic.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

して、2016 年の The DAO 事件では Reentrancy 脆弱性が悪用され、約 7000 万米ドルの暗号資産が流出した [4]。

ブロックチェーンの特徴の一つである不変性からスマートコントラクトは一度公開されると容易に変更できないため、脆弱性が発見された場合でも迅速な修正が困難である。そのため、安全なコントラクト開発の重要性が指摘されており[*1]、脆弱なスマートコントラクトを無害なものに修正する技術は重要な研究課題となっている [5]。さらに、スマートコントラクトでは脆弱性を修正する際に、その実行に掛かる仮想通貨の消費も抑えなければならない [6]。このように、スマートコントラクトの脆弱性修正には、一般的なプログラムの修正とは異なる難しさが存在する。

上述した課題に関して、大規模言語モデル（LLM）を活用したスマートコントラクトの脆弱性修正技術について議論されている [7,8]。LLM によるコード修正では、修正対象箇所を限定することで、不要なコード領域を探索する必要がなくなり、成功率向上することが示されている [9]。

しかしながら、既存の手法では、脆弱性の原因箇所を分析できない場合がある [9]。特に一部の脆弱性は関数呼び出しや状態遷移などの構造的特徴に依存するため [10]、ソースコードを単純なトークン列として扱うだけで LLM が脆弱性を認識できるとは限らない [11]。ソースコードを抽象構文木（AST）などのグラフ構造として表現し、グラフニューラルネットワーク (GNN) によって脆弱性を特定する技術 [10] もあるが、GNN による脆弱性の分析と LLM によるコード修正は独立に議論されており、それらを組み合わせるだけで修正能力が向上するとは限らない。GNN が抽出した特徴を LLM がそのまま解釈できる保証はなく、構造情報の付加はプロンプト長を増大させることで修正性能を損なう可能性もある。したがって、GNN が抽出した構造的特徴が LLM によるスマートコントラクトの脆弱性の修正能力の向上に寄与するかは非自明である。

そこで本研究では、以下の問いに答える: **GNN によって分析されたソースコードの構造的特徴は、LLM によるスマートコントラクトの脆弱性修正性能向上に寄与するか?**

1.2 貢献

本稿の貢献は以下の二点である。一点目は、スマートコントラクトの脆弱性修正に関して、GNN と LLM を統合した自動修正手法 EthReliever を提案する。EthReliever では、スマートコントラクトの AST を入力とした GNN により脆弱性に関連する箇所を分析し、その結果を LLM へ入力することで, ソースコードの構造的特徴を考慮した脆弱性の修正が可能となる。関連研究と EthReliever との比較について、表 1 に示す。EthReliever は著者の知る限り, 脆弱性の検知とソースコードの構造的特徴の抽出も含めてコードを修正する初の方式である。（詳細は 4 節参照.）

表 1: 関連研究と EthReliever の比較

手法	脆弱性検知	コード修正	ソースコードの構造的特徴抽出
SCRepair [6]		✓	
sGuard+ [5]	✓	✓	
LogicRepair [7]	✓	✓	
BugSweeper [10]	✓		✓
SolAgent [8]	✓	✓	
EthReliever	✓	✓	✓

二点目は、Solidity で記述されたスマートコントラクトを対象とした評価を行い、GNN によって抽出されたソースコードの構造的特徴が、LLM による脆弱性修正性能の向上に寄与することを示す。公開データセット上で既存の脆弱性修正手法 [5,8] と比較したところ, Gas の変化量を実用的な範囲に抑えたまま、脆弱性の修正が可能になっていることを確認する。これは EthReliever が GNN を通じてソースコードの構造的特徴を効果的に抽出できるようになったことが性能改善に寄与していることを示す。実験により得られた知見として、EthReliever では GNN を導入したことにより、脆弱性の修正に掛かる LLM のトークン消費量を、LLM を複数組み合わせるような方法よりも抑えられることを示す。また、脆弱性の修正時に EthReliever が内包する LLM の処理を、修正したコードに対する脆弱性の検知を踏まえたうえで繰り返し実行することで、脆弱性修正を大幅に改善できることを示す。（詳細は 5 節参照.）

2. 準備

2.1 スマートコントラクト

Ethereum [1] はブロックチェーン上に展開されるプログラムであるスマートコントラクトを、Ethereum の P2P ネットワーク参加者により管理される Ethereum 仮想マシン（EVM）を通じて提供する。スマートコントラクトは一度ブロックチェーン上に展開されると変更できず、また、その実行によって誰もが同じ結果を得られる。Ethereum のスマートコントラクトは Solidity という高水準言語でよく実装され、Solidity には主要なコンパイラバージョンとして v0.1 から v0.8 までの 8 つがある。各スマートコントラクトにはコントラクトアドレスが割り当てられ、そのコントラクトアドレスを通じて暗号通貨 Ether を受け取ることで関数が実行される。各スマートコントラクトの実行コストとして消費される暗号通貨は Gas と呼ばれ、EVM を持つ参加者に対して関数実行の対価として支払われる。

2.2 大規模言語モデル（LLM）

大規模言語モデル（Large Language Model: LLM）は、大量のテキストデータを用いて事前学習された、Transformer

[*1] https://ethereum.org/developers/docs/smart-contracts/security

© 2026 Information Processing Society of Japan

を基盤とする言語モデルである [12]。LLM は、与えられた入力トークン列に対して後続のトークンを逐次予測することでテキストを生成し [12]、自然言語処理をはじめソースコードの生成や修正など幅広いタスクに利用される [13]。具体的には、LLM の出力は入力テキストであるプロンプトによって制御され、解くべきタスクの説明や参考情報をプロンプトに含めて LLM に与えることで、期待の出力を得ることができる [12]。代表的な LLM としては、OpenAI が開発した GPT [14] 系のモデルが挙げられる。

2.3 グラフニューラルネットワーク（GNN）

グラフニューラルネットワーク（Graph Neural Network: GNN）は、ノードとエッジから構成されるグラフ構造データを処理するために設計された深層学習モデルの一種であり、ノード間の関係パターンを学習し、ノード分類、グラフ分類、グラフ生成などのタスクを実行する [15]。代表的な GNN には、グラフ畳み込みネットワーク（GCN）[16]、グラフアテンションネットワーク（GAT） [17] などがある。GCN は、隣接するノードの特徴を使用して畳み込み演算を実行し、ノードの特徴を更新し、GAT は、アテンションメカニズムを使用して、隣接するノードに異なる重み付けを行い、ノードの特徴をより細かく更新する。

2.4 関連研究

スマートコントラクトに含まれる脆弱性を発見するため、様々な対策手法が提案されている。これまでルールベースでの検知手法 [18] が提案され、とくに Slither [18] は高精度かつ高速に検知できる。しかし、これらは未知の脆弱性や複雑な実行経路に依存する脆弱性には限界がある [13]。

この課題に対し、機械学習ベースでの脆弱性検知手法が提案されている [10,19]。これらの手法では、スマートコントラクトを学習データとして、脆弱性の有無や種類を検知する。とくに GNN を活用した技術 [10, 19] では、ソースコードを AST などグラフ構造として入力し、それらの構造的特徴を学習する。脆弱性の判定に加え、脆弱性に関連するソースコード行や AST ノードを特定する技術 [19] もあり、また、LLM による脆弱性検知 [9] も報告されている。

一方で、自動コード修正（Automated Program Repair: APR）と呼ばれる、プログラムに含まれる不具合を自動的に修正する技術が注目されている [13]。APR はスマートコントラクトに限らず C 言語など一般的なプログラムを対象としており [13], スマートコントラクトを対象とした研究では事前に定義した修正ルールが適用されている [5,6]。なお、上述した手法は脆弱性が限定され、未知の修正パターンへの対応が困難である。このため、LLM を活用したコード修正手法 [7,8] も議論されている。本稿ではルールベースと LLM ベースそれぞれの最新成果として sGuard+ [5] と SolAgent [8] と比較実験を行う。

3. 問題定義

本稿では、脆弱性を含むスマートコントラクトのソースコードを自動的に修正する問題を扱う。修正対象のソースコードは $SC = \{L_1, ..., L_n\}$ と表すことができる。ここで L_i $(1 \leq i \leq n)$ はソースコードを構成する各行を表し、n はその総行数である。この時、本稿の目的は、SC から脆弱性を除去した修正後ソースコード $SC' = \{L'_1, ..., L'_{n'}\}$ を生成することである。ここで L'_j は修正後の各行、n' は修正後の総行数を表す。修正後ソースコード SC' は次の三つの要件を満たすものとする。第一に、SC' は脆弱性を含まない（脆弱性の除去）。第二に、任意の正当なトランザクション Tx について、Tx が SC 内の関数 F を正常に呼び出す場合、SC' 内には同等の機能を有する関数 F' が存在し、F と同一または等価な結果を返す（機能の等価性）。第三に、SC' の実行速度とガスコストについて、SC と比較して著しく劣化しない（実行効率の維持）。

次に、本研究が対象とする脆弱性として Reentrancy, tx.origin, Unchecked low-level, Tautology について述べる。Reentrancy は、外部コントラクトの呼び出し後に状態を更新する実装において、状態更新前に再帰的な呼び出しを受けることで状態の不整合が生じる脆弱性である。tx.origin は、tx.origin を用いて呼び出し元を認証する実装において、正規利用者を経由した攻撃によって認証が迂回される脆弱性である。Tautology は、常に真または常に偽となる条件式を用いることで、意図した検査が機能しない脆弱性である。Unchecked low-level は、call() や delegatecall() といった失敗してもトランザクションをロールバックせず、代わりにブール値を返す関数を使用する際に、戻り値をチェックしないことで引き起こされる脆弱性である。

4. EthReliever の設計

本節では EthReliever を提案する。まず EthReliever の全体像を述べたのち、その主なモジュールとして脆弱性分析部、コード修正部、評価部をそれぞれ述べる。

4.1 システム全体像

EthReliever の全体像を図 1 に示す。EthReliever は、脆弱性を含むスマートコントラクトのソースコード SC を入力として受け取り、脆弱性を除去した修正済みソースコード SC' を出力する。EthReliever は、脆弱性分析部、コード修正部、評価部の 3 つの機能から構成される。まず脆弱性分析部では、脆弱性を含むソースコード SC を入力とし、脆弱性の原因箇所および脆弱性の種類に関する分析情報 X を生成する。次にコード修正部では、ソースコード SC と分析情報 X を用いて LLM への入力プロンプトを生成し、修正後コード SC' を生成する。最後に評価部では、生成した修正後コード SC' に対して静的解析を行い、脆

© 2026 Information Processing Society of Japan

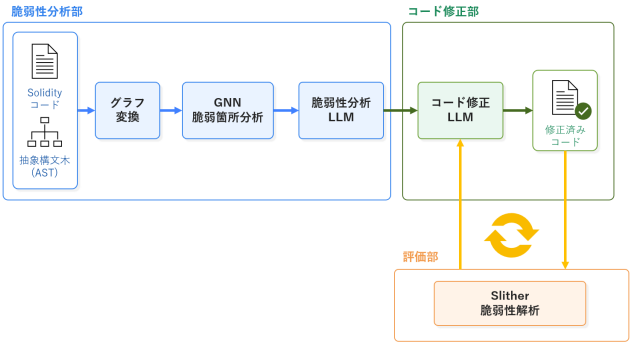


図 1: EthReliever の全体像

弱性が除去されたかを確認する。必要に応じてコード修正部へ再入力し、再修正を実施する。EthReliever のアルゴリズムを付録 A.2 に示す。

4.2 脆弱性分析部

4.2.1 GNN による脆弱箇所の分析

本稿では、ソースコードの構造情報を扱うため、Solidity コンパイラ solc [*2]を用いて、ソースコード SC を抽象構文木（AST）T_{SC} へ変換する。T_{SC} はソースコードの構文構造を木構造として表現し、関数定義や変数宣言、条件分岐などの構文要素を階層的に保持する。さらに、AST をグラフ構造として表現したグラフ $G_T = \{V, E\}$ を構築する。ここで $V = \{v_1, ..., v_m\}$ は G_T を構成するノードの集合であり、AST を構成する木構造をその階層単位で区切ったものをノード v_i とする。v_i はノードに含まれる構文要素の種類、対応するソースコード内の記述、位置の情報を AST から抽出する。$E \subseteq V \times V$ は AST の階層構造をノード間の親子関係として表したエッジの集合である。

EthReliever では、脆弱性の種類がラベル付けされたスマートコントラクトのソースコードで学習済みの GNN f_θ を利用する。この GNN はグラフ G_T を入力とし、入力されたコントラクトが各脆弱性の種類にどの程度該当するか推論し、該当する可能性のある脆弱性の種類とその確信度を表すスコアを出力する。すなわち、対象とする脆弱性の種類の集合を $\mathcal{C}$ とすると、GNN は $\mathbf{s} = f_\theta(G_T), \quad \mathbf{s} = \{s_c \mid c \in \mathcal{C}\}$ を出力する。ここで s_c は、入力コントラクトが脆弱性の種類 c に該当するスコアである。

次に、脆弱性の原因箇所の詳細な分析として、グラフ G_T の各ノードをマスクしたグラフ $G_T^{(-i)}$ を生成し、再度それぞれのスコアを推論する。ノード v_i をマスクした際のスコアを $\mathbf{s}^{(i)}$ とすると、マスク前のスコア $\mathbf{s}$ からの減少幅 $\Delta_i = ||\mathbf{s} - \mathbf{s}^{(i)}||$ が大きいノードは脆弱性に寄与すると考えられ、EthReliever は減少幅が最大となるノード $v_{vul} \leftarrow \arg\max_{v_i} \Delta_i$ が脆弱性の原因箇所候補と判断する。

最後に、該当する可能性のある脆弱性の種類とそのスコア $\mathbf{s}$、脆弱性の原因箇所候補となる AST ノード v_{vul}、およびその寄与度 Δ_{vul} から GNN から得られる脆弱性分析情報として $X = (\mathbf{s}, \Delta_{vul}, v_{vul})$ を構築する。この情報 X は、後続の LLM による脆弱性分析に入力される。AST 上のノード間関係を考慮することで、Reentrancy など構造的特徴に依存する脆弱性の高精度な分析が期待できる。

*2 https://github.com/argotorg/solidity/

4.2.2 LLM による脆弱性分析

GNN から得られた X を用いて、脆弱性分析用 LLM（L_{ana}）が脆弱性の原因および修正方針を生成する。具体的には、$X' = L_{ana}(SC, X)$ とする。ここで、X' は脆弱箇所の説明および修正方針を含む自然言語形式の分析結果である。この分析結果は後続のコード修正部に入力される。

4.3 コード修正部

コード修正部では、ソースコード SC と脆弱性分析結果 X' を基にコード修正用プロンプト P を生成する。プロンプト生成処理は、$P = Prompt(SC, X')$ としてあらわすことができる。生成されるプロンプトには脆弱箇所の説明、想定される脆弱性種類、修正方針を含める。生成したプロンプト P をコード修正用 LLM（L_{rep}）へ入力し、$SC' = L_{rep}(P)$ を生成する。

4.4 評価部

評価部では、コード修正部が生成した修正後コード SC' が問題定義の要件を満たすかを検証する。具体的には、静的解析ツールにより脆弱性リスト R を取得し、R に脆弱性が残存しない場合を修正成功とみなす。一方、脆弱性が残存する場合、評価部は再修正 Loop を実行する。再修正 Loop では、元のソースコード SC、および静的解析ツールから得られた脆弱性情報 R を用いて再修正用プロンプト $P' = Prompt(SC, R)$ を生成し、コード修正用 LLM へ再入力して修正を試みる。この処理を、修正が成功するか、あらかじめ定めた最大反復回数に達するまで繰り返す。

5. 実験と考察

5.1 目的

本節では大きく二つの評価を行う。第一に、EthReliever を既存のスマートコントラクト脆弱性修正手法と比較する。構造的特徴を活用する EthReliever が既存手法に対して高い修正性能を示すことを確認することで、ソースコードの構造的特徴が修正性能の向上に寄与するかを評価する。

第二に、EthReliever を構成する各機能が修正性能へ与える影響を明らかにするため、Ablation study を行う。EthReliever は脆弱性分析部、コード修正部、評価部から構成されるため、これらの機能を段階的に取り除いた構成、あるいは置き換えた構成を比較することで、どの機能が修正性能の向上に有効であるかを分析する。具体的には、脆弱性分析部を持たずソースコードのみを LLM へ入力する構成と比較することで、脆弱性分析部が生成する分析情報

© 2026 Information Processing Society of Japan

をコード修正部へ与えることの効果を評価する。また、脆弱性分析部における脆弱箇所の分析方法として GNN を用いる構成とその他の構成を比較することで、GNN による構造的特徴の抽出が修正性能へ与える影響を評価する。さらに、評価部を用いず一度のみ修正を行う構成と、評価部による再修正を反復する構成を比較することで、静的解析のフィードバックを用いた再修正の有効性を評価する。

5.2 実験設定

本節では、既存手法との比較および Ablation study の実験設定を述べる。いずれの実験においても、コード修正部には OpenAI の公開モデルである o4-mini（API version: 2024-12-01-preview）、脆弱性分析部には gpt-5-mini（API version: 2024-06-01）を、ファインチューニングを行わずに利用した。また、脆弱性分析部の GNN には、グラフ畳み込みネットワーク（GCN）を用いた。

5.2.1 評価用データセット

評価にはそれぞれ脆弱性の種類・実コードとしての複雑さ・Solidity バージョンの異なる 2 種類のデータセットを用いた。not-so-smart-contracts[*3] は、Trail of Bits による脆弱性サンプルであり、25 件の Solidity 0.4.x 系コントラクトを含む。Trapdoor-scam[*4] は、実際の詐欺コントラクトデータセットのうち行数の少ない 100 件であり、Solidity 0.7.x〜0.8.x 系のコントラクトである。なお、各データセットに含まれるソースコードの複雑さを把握するため、LoC（Lines of Code）を測定した。LoC 算出の際は、コメントアウト、並びに空白行を除外した実行可能な行数をカウントし、not-so-smart-contracts、Trapdoor-scam の平均 LoC はそれぞれ 102.5 行、119.5 行であった。

5.2.2 既存手法との比較

5.1 節で述べた第一の評価として、EthReliever を既存のスマートコントラクト脆弱性修正手法である sGuard+ [5] および SolAgent [8] と比較する。ただし、評価部の静的解析ツールは Slither [18] を使用し、再修正 Loop は最大 3 回（$N = 3$）までとした。SolAgent はスマートコントラクトの要件が定義された仕様書も入力に取るが、本稿では SolAgent の単純な脆弱性修正能力を比較するため、仕様書からスマートコントラクトの生成は実施せず、脆弱性修正の処理のみを実行した。なお、本比較における EthReliever の値は、5.2.3 節で述べる実験条件のうち、脆弱性分析部に GCN と LLM を併用し、ソースコードのみをプロンプト入力とする構成（後述の表記方法における「GCN+LLM/LLM」）において、評価部による再修正 Loop を最大 3 回実施した時点の値である。

5.2.3 Ablation study

5.1 節で述べた第二の評価として、脆弱性分析部およびコード修正部の構成を変更した複数の条件を比較する Ablation study を行う。各表内の実験条件は「脆弱性分析部/コード修正部」の形式で表記し、例えば「LLM/LLM」は脆弱性分析部および修正部とも LLM を使用する構成を、「GCN/LLM」は GCN で脆弱性箇所を分析した上で LLM でコード修正を行う構成を示す。また、各表の「プロンプト入力」列は、脆弱性分析部およびコード修正部へ渡す情報として、脆弱性分析用およびコード修正用プロンプト含めたソースコードおよび構造情報を示し、ソースコード (SC)、AST(T_{SC})、グラフ (G_T) の組み合わせで表記する。比較条件は大きく以下の 3 つの観点から構成される。

第一に、脆弱性分析部の有無を評価する。単一 LLM 構成（-/LLM）と、脆弱性分析部を導入した構成を比較することで、脆弱性分析工程そのものが修正性能へ与える影響を評価する。ここで単一 LLM 構成（-/LLM）では、図 1 の脆弱性分析部を経由せず、SC のみをコード修正部へ直接入力する。第二に、プロンプトとして LLM へ入力する構造情報の有無を評価する。入力情報として、SC のみ、SC + AST(T_{SC})、SC +グラフ (G_T)、および SC + AST(T_{SC}) +グラフ (G_T) の四種類を比較することで、ソースコードの構造的特徴が修正性能へ与える影響を評価する。ただし、SC、AST(T_{SC})、グラフ (G_T) はいずれもトークン列として利用する。第三に、脆弱性箇所の分析手法の違いを評価する。脆弱性分析部として LLM、GCN、GCN + LLM を比較することで、脆弱箇所分析手法による影響を評価する。GCN のみを用いる構成（GCN/LLM）では、図 1 の脆弱性分析 LLM を経由せず、GCN が分析した脆弱ノードの情報を直接コード修正部へ入力する。

以上の全ての条件に対し、評価部による再修正 Loop の有無を比較した。再修正 Loop では Slither による検知結果を再度コード修正部へ入力し、最大 3 回まで再修正する。

5.2.4 評価指標

EthReliever の脆弱性修正性能は、脆弱性修正の成功率で評価した。具体的には、Slither [18] による解析で impact が Medium 以上の脆弱性の検知件数が 0 件となった場合を修正成功とみなす。ここで impact とは、検知された脆弱性による影響の大きさを Slither が分類した指標であり、High、Medium、Low、Informational、Optimization の 5 段階で与えられる。本稿では、資産の流出や機能停止など実害に直結し得る High および Medium を修正の対象とし、コーディング規約や最適化に関する指摘が中心となる Low 以下は対象外とした。このとき、成功率は、$SuccessRate = N_{success}/N_{all}$ である。ここで、$N_{success}$ は上記の基準で修正に成功したサンプル数、N_{all} は評価対象総サンプル数である。以降では評価部による再修正を行わず 1 回のみ修正を実施した時点を One-pass と言い、One-pass 時点の成功率、及び、再修正 Loop を最大 3 回まで実施した後の成功率をそれぞれ算出した。

[*3] https://github.com/crytic/not-so-smart-contracts
[*4] https://github.com/bsdp2023/trapdoor-scam

© 2026 Information Processing Society of Japan

また、既存研究 [8] を参考に、以下の 3 指標も算出した。平均消費トークン数は LLM が脆弱性修正に消費したトークン数である。Gas 変化量は、コンパイラの静的解析により算出した、修正前後のデプロイ時ガスコストの変化率である。ただし、ガスコストの見積もりにはコンパイルが必要であるため、修正後コードがコンパイル可能であったスマートコントラクトのみを母数とする。脆弱性改善率は、静的解析ツールによる検知件数がどれだけ減少したかを $1 - N_{after}/N_{before}$ で表した値であり、N_{before} は修正前、N_{after} は修正後の検知件数である。なお、この値はスマートコントラクトごとに算出した比の平均ではなく、データセット全体の検知件数の合計に対する比として算出する。

5.3 実験結果

5.3.1 既存手法との比較

表 2 に、EthReliever、sGuard+ [5] と SolAgent [8] の比較結果を示す。EthReliever は、いずれのデータセットにおいても既存手法を上回る成功率を達成した。EthReliever の成功率は、not-so-smart-contracts で 72.0%、Trapdoor-scam で 92.0% であった。これに対し、テンプレートベースの修正手法である sGuard+ は、いずれのデータセットでも成功率が 0.0% であり、脆弱性を除去できなかった。これは、sGuard+ が対応する脆弱性の種類が限定されており、本実験で用いたデータセットに含まれる多様な脆弱性に対応できなかったためと考えられる。また、SolAgent の成功率は、not-so-smart-contracts で 52.0%、Trapdoor-scam で 48.0% であった。EthReliever はこれらをいずれも上回っており、特に Trapdoor-scam では SolAgent の 48.0% に対して 92.0% と大きく上回った。同様に、脆弱性改善率についても EthReliever は、not-so-smart-contracts で 56.9%、Trapdoor-scam で 92.1% と高い値を示した。

すなわち、GCN により構造的特徴を活用することで、EthReliever は既存手法より高い修正性能を示している。

5.3.2 Ablation study

表 3 は評価部による再修正を行わない One-pass 時点の結果を、表 4 は再修正 Loop を最大 3 回まで実施した時点の結果を、いずれもデータセットごとに示している。

まず、表 3 から脆弱性分析部の有無による修正性能を比較する。脆弱性分析部を持たない単一 LLM 構成 (-/LLM) の 成功率は、not-so-smart-contracts で 44.0%、Trapdoor-scam で 18.0% であった。脆弱性分析部を導入した構成では、not-so-smart-contracts で 52.0-56.0%、Trapdoor-scam で 19.0-40.0% であり、特に not-so-smart-contracts および Trapdoor-scam では単一 LLM 構成を上回った。

次に、プロンプトとして LLM へ入力する構造情報の違いによる修正性能を比較する。ソースコードのみを入力した構成 (LLM/LLM, SC) の One-pass 成功率は、not-so-smart-contracts で 56.0%、Trapdoor-scam で 37.0% であった。これに AST を併用した構成 (LLM/LLM, SC, T_{SC}) ではそれぞれ 52.0%、29.0% と低下し、グラフを併用した構成 (LLM/LLM, SC, G_T) では 56.0%、40.0% と同等以上である。また、AST およびグラフを併用した構成 (LLM/LLM, SC, T_{SC}, G_T) ではそれぞれ 52.0%、19.0% であった。

さらに、脆弱箇所の分析手法の違いによる修正性能を比較する。脆弱箇所分析に GCN を用いた構成 (GCN/LLM、GCN+LLM/LLM) の平均トークン数は約 4600-17000 であり、AST およびグラフを併用した構成より少ない。One-pass 成功率は、GCN を用いた構成が not-so-smart-contracts で 48.0-52.0%、Trapdoor-scam で 20.0-30.0% であり、いずれも単一 LLM 構成を上回る。Trapdoor-scam では GCN/LLM の Loop 成功率が 99.0% と最も高い。

最後に、評価部による再修正 Loop による修正性能の向上を評価する。全データセット・全条件において、One-pass 成功率から Loop 成功率への上昇が確認された。とくに Trapdoor-scam の単一 LLM 構成 (-/LLM) では 18.0% から 98.0% へ上昇した。

以上の結果を、5.3.1 節で示した既存手法との比較と併せて整理する。既存手法との比較では、GCN によって構造的特徴を抽出する EthReliever が、いずれのデータセットにおいても高い成功率を示した。また、表 3 の Ablation study では、構造的特徴をグラフとして与えた構成および脆弱性分析に GCN を用いた構成が、構造的特徴を与えない単一 LLM 構成の One-pass 成功率を、いずれのデータセットにおいても上回った。これらは、ソースコードの構造的特徴を GNN が分析することで、LLM による脆弱性修正に寄与することを示している。

5.4 考察

本節では、5.3.1 節および 5.3.2 節で示した実験結果をもとに、構造的特徴を LLM へ与える形式、構造的特徴の抽出に伴うトークン効率、および評価部による再修正 Loop の有効性という三つの観点から考察する。

まず、構造的特徴をどのような形式で LLM に与えるかが重要である。表 3 に示したように、AST をトークン列としてそのままプロンプトに含めた構成では、ソースコードのみを与えた構成と比べて One-pass 成功率がいずれのデータセットでも低下した。この要因としては二点が考えられる。第一に、AST はノードの識別子やソースコード上の位置といった、修正の判断には直接寄与しない属性を大量に含むため、プロンプト長がソースコードのみの構成の数倍から十数倍に達し、本来注目すべき脆弱箇所の記述が長大な木構造の中に埋没することである。第二に、AST は構文の階層関係を表すにとどまるため、Reentrancy のように外部呼び出しと状態更新の順序関係に依存する脆弱性については、その関係を LLM 自身が木構造から読み取らなければならないことである。すなわち、構造的特徴を与え

© 2026 Information Processing Society of Japan

表 2: EthReliever と既存手法（sGuard+, SolAgent）の性能比較

手法	データセット	成功率	平均消費トークン数	Gas 変化量	脆弱性改善率
sGuard+ [5]	not-so-smart-contracts	0.0%	–	+3.0%	-2.6%
	Trapdoor-scam	0.0%	–	+0.0%	0.0%
SolAgent [8]	not-so-smart-contracts	52.0%	2360	+11.4%	13.9%
	Trapdoor-scam	48.0%	3965	+8.9%	42.6%
EthReliever	not-so-smart-contracts	72.0%	12883	+18.0%	56.9%
	Trapdoor-scam	92.0%	19364	+6.9%	92.1%

表 3: Ablation study（One-pass 時点）

データセット	実験条件	プロンプト入力	成功率	平均トークン数	Gas 変化量	脆弱性改善率
not-so-smart-contracts	-/LLM	SC	44.0%	3929	+7.0%	33.3%
	LLM/LLM	SC	56.0%	8722	+21.3%	31.9%
	LLM/LLM	SC, T_{SC}	52.0%	32855	+22.3%	30.6%
	LLM/LLM	SC, G_T	56.0%	15714	+15.4%	29.2%
	LLM/LLM	SC, T_{SC}, G_T	52.0%	36447	+28.9%	30.6%
	GCN/LLM	SC	48.0%	4646	+5.8%	33.3%
	GCN+LLM/LLM	SC	52.0%	9946	+16.6%	34.7%
Trapdoor-scam	-/LLM	SC	18.0%	7677	+3.5%	5.0%
	LLM/LLM	SC	37.0%	14670	+5.0%	8.9%
	LLM/LLM	SC, T_{SC}	29.0%	130724	+3.6%	3.0%
	LLM/LLM	SC, G_T	40.0%	25460	+4.8%	28.7%
	LLM/LLM	SC, T_{SC}, G_T	19.0%	141212	+1.9%	-5.0%
	GCN/LLM	SC	20.0%	8446	+3.5%	-5.0%
	GCN+LLM/LLM	SC	30.0%	16872	+3.0%	5.0%

表 4: Ablation study（再修正 Loop 後）

データセット	実験条件	プロンプト入力	成功率	平均トークン数	Gas 変化量	脆弱性改善率
not-so-smart-contracts	-/LLM	SC	80.0%	7597	+10.1%	62.5%
	LLM/LLM	SC	72.0%	11608	+22.4%	45.8%
	LLM/LLM	SC, T_{SC}	60.0%	34602	+24.6%	30.6%
	LLM/LLM	SC, G_T	68.0%	16934	+21.7%	26.4%
	LLM/LLM	SC, T_{SC}, G_T	68.0%	38264	+30.2%	29.2%
	GCN/LLM	SC	68.0%	8954	+7.3%	62.5%
	GCN+LLM/LLM	SC	72.0%	12883	+18.0%	56.9%
Trapdoor-scam	-/LLM	SC	98.0%	10985	+7.0%	98.0%
	LLM/LLM	SC	93.0%	16931	+7.5%	93.1%
	LLM/LLM	SC, T_{SC}	84.0%	132840	+6.6%	84.2%
	LLM/LLM	SC, G_T	98.0%	27558	+8.0%	98.0%
	LLM/LLM	SC, T_{SC}, G_T	85.0%	143670	+6.2%	85.1%
	GCN/LLM	SC	99.0%	11732	+7.3%	99.0%
	GCN+LLM/LLM	SC	92.0%	19364	+6.9%	92.1%

ること自体ではなく、LLM が解釈しやすい形に整えて与えられていないことが問題であると考えられる。したがって、木構造をそのまま列挙するのではなく、ノード間の関係を保ったグラフとして与えるか、さらにそのグラフから脆弱性に寄与する箇所のみを抽出して簡潔に与えることが有効であると考えられる。以上より、構造的特徴は AST をそのまま提示するのではなく、グラフ表現および GCN による抽出を通じて活用していくべきであるといえる。

© 2026 Information Processing Society of Japan

さらに、GCN を用いた構成は、消費トークン数が少なく、利用コストや処理時間が効率的といえる。表 3 に示したように、GCN と LLM を併用して脆弱箇所を分析した構成（GCN+LLM/LLM）は、AST を併用した構成と同等以上の One-pass 成功率を、その 3 分の 1 から 8 分の 1 程度のトークン数で達成している。すなわち、GCN による構造的特徴の抽出は、修正性能を維持しながらトークン消費を抑えることができる。LLM の利用コストや処理時間はトークン数に依存するため、少ないトークン数で同等以上の修正性能を達成できる GCN 構成はコストパフォーマンスが高く、実用上優れているといえる。

最後に、評価部による再修正 Loop は、修正性能の向上に一貫して有効である。表 4 の Loop 成功率を見ると、全てのデータセット・全ての条件において、表 3 の One-pass 成功率を上回る。また、表 3 では脆弱性改善率が負の値を示す条件がある。これは one-pass 時点の修正では新たな脆弱性が生じる可能性を示すが、表 4 ではこれらの条件も正の値に転じている。これらは、LLM による修正が新たな脆弱性を導入し得ること、およびそれを静的解析ツールのフィードバックによって段階的に是正できることを示しており、修正結果の検証と再修正の重要性を裏づけている。

6. 結論

本稿では、GNN による脆弱箇所特定と LLM によるコード修正を統合したスマートコントラクト脆弱性修正手法 EthReliever を提案した。二種類の公開データセットで計 125 件の脆弱性を含むスマートコントラクトを用い評価したところ、EthReliever は sGuard+ [5] および SolAgent [8] と比べて高い成功率で脆弱性を修正できることを示した。Ablation study によると、これは AST を平易に与えるよりも, GNN がそれをソースコードの構造的特徴として解釈することで、LLM による脆弱性の修正性能に寄与するためである。特に、GNN を用いることで、高い成功率を維持しながら LLM が消費するトークン数を大幅に削減できる。また、EthReliever の評価部において再修正に関する Loop 処理を導入することで、全ての実験条件で脆弱性修正の成功率が向上した。今後は、EthReliever を AI エージェントにより制御することで、入力されるスマートコントラクトの構成が柔軟に変化したとしても、脆弱性の修正を効果的に行える方法を模索する。

謝辞. 本研究の一部は、JST さきがけ (JPMJPR23P6) の助成を受けた。

参考文献

[1] Wood, G.: Ethereum: A secure decentralised generalised transaction ledger Byzantium Version, `ethereum.github.io/yellowpaper/paper.pdf` (2022).

[2] Ghosh, A., Lavanya, Hassija, V., Chamola, V. and El Saddik, A.: A Survey on Decentralized Metaverse Using Blockchain and Web 3.0 Technologies, Applications, and More, *IEEE Access*, Vol. 12, pp. 146915–146948 (2024).

[3] CERTIK: Hack3d: The Web3 Security Report 2025, https://www.certik.com/blog/hack3d-the-web3-security-report-2025 (2025). Access date: 2026/08/20.

[4] GeeksforGeeks: What was the DAO Hack?, `https://www.geeksforgeeks.org/ethical-hacking/what-was-the-dao-hack/` (2025). Access date: 2026/07/28.

[5] Gao, C., Yang, W., Ye, J., Xue, Y. and Sun, J.: sGuard+: Machine Learning Guided Rule-Based Automated Vulnerability Repair on Smart Contracts, *ACM Trans. Softw. Eng. Methodol.*, Vol. 33, No. 5 (2024).

[6] Yu, X. L., Al-Bataineh, O., Lo, D. and Roychoudhury, A.: Smart contract repair, *ACM Transactions on Software Engineering and Methodology (TOSEM)*, Vol. 29, No. 4, pp. 1–32 (2020).

[7] Ai, M., Wang, K., Wang, H., Zheng, C. and Zhang, Y.: LogicRepair: An Empirical Study on Automated Repair of Smart Contract Logic Vulnerabilities Based on Large Language Models, p. 261 – 266 (2025).

[8] Chen, W., Peng, Z., Yin, X., Ni, C., Ying, C., Xie, B. and Luo, Y.: SolAgent: A Specialized Multi-Agent Framework for Solidity Code Generation (2026).

[9] Yu, Z., Guo, Z., Wu, Y., Yu, J., Xu, M., Mu, D., Chen, Y. and Xing, X.: PATCHAGENT: A Practical Program Repair Agent Mimicking Human Expertise, *Proc. of USENIX Security 2025*, pp. 4381–4400 (2025).

[10] Lee, U., Chung, C., Lee, J. and Moon, S.-M.: BugSweeper: Function-Level Detection of Smart Contract Vulnerabilities Using Graph Neural Networks, *Proc. of AAAI 2026*, Vol. 40, No. 1, pp. 570–578 (2026).

[11] Wang, R., Li, Z., Gao, C., Wang, C., Xiao, Y. and Wang, X.: SPVR: syntax-to-prompt vulnerability repair based on large language models, *Automated Software Engineering*, Vol. 33, No. 1, p. 38 (2026).

[12] Zhao, W. X., Zhou, K., Li, J., Tang, T., Dong, Z., Hou, Y., Zhang, B., Min, Y., Zhang, J., Liu, P. et al.: A survey of large language models, *Frontiers of Computer Science*, Vol. 20, No. 12, p. 2012627 (2026).

[13] Hu, Y., Liu, Z., Shu, K., Guan, S., Zou, D., Xu, S., Yuan, B. and Jin, H.: SoK: Automated Vulnerability Repair: Methods, Tools, and Assessments, *Proc. of USENIX Security 2025*, USENIX, pp. 4421–4440 (2025).

[14] Radford, A., Narasimhan, K., Salimans, T., Sutskever, I. et al.: Improving language understanding by generative pre-training (2018).

[15] Luo, X., Zhao, Y., Qin, Y., Ju, W. and Zhang, M.: Towards semi-supervised universal graph classification, *IEEE Transactions on Knowledge and Data Engineering*, Vol. 36, No. 1, pp. 416–428 (2023).

[16] Kipf, T. N. and Welling, M.: Semi-supervised classification with graph convolutional networks, *arXiv preprint arXiv:1609.02907* (2016).

[17] Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y. et al.: Graph attention networks, *stat*, Vol. 1050, No. 20, pp. 10–48550 (2017).

[18] Feist, J., Grieco, G. and Groce, A.: Slither: A Static Analysis Framework for Smart Contracts, *Proc. of WETSEB 2019*, IEEE, p. 8–15 (2019).

[19] Wang, B., Li, S., Yuan, X., Xie, X., Wang, J., Li, T. and Wang, W.: ContractScanner: Detecting and Localizing Vulnerabilities of Smart Contracts via Graph-Based Semantic Modeling of Source Code, *IEEE Transactions on Network Science and Engineering* (2025).

© 2026 Information Processing Society of Japan

付　　録

A.1　入力プロンプト例

脆弱性分析部、コード修正部、評価部で用いたプロンプトの例を図 A·1〜A·3 に示す。

```
下記のSolidityコードには脆弱性が含まれている可能性があります。
参考データを確認して、どのような脆弱性があるかについて分析し、
以下のJSON形式で発見した脆弱性を列挙してください。

# 参考データ
<ソースコード/AST/グラフ/GNN分析結果>
…
# 出力JSON形式
…
```

図 A·1: 脆弱性分析 LLM への入力プロンプト例

```
下記のSolidityコードには脆弱性が含まれている可能性があります。
参考データを確認して、脆弱性を解消するようにSolidityコードを修
正してください。

# 参考データ
<ソースコード/AST/グラフ/GNN分析結果>
<脆弱性分析LLMの出力JSON>
…
```

図 A·2: コード修正 LLM への入力プロンプト例

```
下記のSolidityコードは、脆弱性修正ツールによる修正後のコード
ですが、静的解析ツール Slither による検査で脆弱性が検知されま
した。
検知結果を確認して、脆弱性を修正してください。

# 対象のSolidityコード
<ソースコード>
# Slitherによる検知結果
<Slither出力>
…
```

図 A·3: 再修正 Loop 時のコード修正 LLM への入力プロンプト例

A.2　EthReliever の詳細アルゴリズム

EthReliever の詳細アルゴリズムをアルゴリズム 1 に示す。

アルゴリズム 1 EthReliever

入力: 脆弱性を含むスマートコントラクト SC
出力: 修正済みスマートコントラクト SC'

function 脆弱性分析部 (SC)
  $T_{SC} \leftarrow \mathrm{solc}(SC)$
  $G_T \leftarrow$ グラフ生成器 (T_{SC})
  $\mathbf{s} \leftarrow f_\theta(G_T)$
  for $i = 1$ **to** m **do**
    $\mathbf{s}^{(i)} \leftarrow f_\theta(G_T^{(-i)})$
    $\Delta_i = ||\mathbf{s} - \mathbf{s}^{(i)}||$
  end for
  $\Delta_{vul} = \max_i \Delta_i$
  $v_{vul} \leftarrow \arg\max_{v_i} \Delta_i$
  $X \leftarrow (\mathbf{s}, \Delta_{vul}, v_{vul})$
  $X' \leftarrow L_{ana}(SC, X)$
  return X'
end function
function コード修正部 (SC, X')
  $P \leftarrow Prompt(SC, X')$
  $SC' \leftarrow L_{rep}(P)$
  return SC'
end function
function 評価部 (SC, SC')
  脆弱性検知リスト $R \leftarrow$ 静的解析ツール (SC')
  for $i = 1$ **to** N **do**
    if $R == \varnothing$ **then**
      break
    end if
    $P' \leftarrow Prompt(SC, R)$
    $SC' \leftarrow L_{rep}(P')$
    $R \leftarrow$ 静的解析ツール (SC')
  end for
  return SC'
end function
$X' \leftarrow$ 脆弱性分析部 (SC)
$SC' \leftarrow$ コード修正部 (SC, X')
$SC' \leftarrow$ 評価部 (SC, SC')
return SC'

© 2026 Information Processing Society of Japan